

```
/*  
  
* BRAILLE HOME AUTOMATION - REMOTE MODULE (braille key pad module)  
  
* - Emergency Logic:  
  
*   - Long press BOTH sensors = Send "ACK" (Confirm)  
*   - Long press ONE sensor  = Send "STOP" (False Alarm)  
  
* - Feedback System:  
  
*   - 'S' = Short beep/vib (400ms)  
*   - 'E' = Long beep/vib (800ms)  
*   - 'C' = 5 short beeps/vibs (correct password)  
*   - 'W' = 5 long beeps/vibs (wrong password)  
*   - 'F' = Continuous short beeps/vibs until ACK/STOP  
  
*/
```

```
#include <WiFi.h>
```

```
const char* ssid = "ESP32-Server";  
const char* password = "12345678";  
const char* serverIP = "192.168.4.1";
```

```
WiFiClient tcpClient;
```

```
// Pins - SINGLE BUZZER AND VIBRATION MOTOR
```

```
const int vibMotor = 12; // Single vibration motor  
const int buzzer = 25;  // Single buzzer  
const int btnPins[] = {5, 17, 16, 4, 2, 15}; // REVERSED ORDER  
const int touchLeft = 32;  
const int touchRight = 33;
```

```
// Globals
```

```
String localBuffer = "";  
bool emergencyMode = false;  
unsigned long emergencyTimer = 0;  
unsigned long lastDebounce = 0;
```

```

const byte brailleMap[] = {
  1, 3, 9, 25, 17, 11, 27, 19, 10, 26, // A-J
  5, 7, 13, 29, 21, 15, 31, 23, 14, 30, // K-T
  37, 39, 58, 45, 61, 53           // U-Z
};

const char brailleChars[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

void setup() {
  Serial.begin(115200);

  // Init Output
  pinMode(vibMotor, OUTPUT);
  digitalWrite(vibMotor, LOW);
  pinMode(buzzer, OUTPUT);
  digitalWrite(buzzer, LOW);

  // Init Input - REVERSED BUTTON ORDER
  for(int i=0; i<6; i++) pinMode(btnPins[i], INPUT_PULLUP);
  pinMode(touchLeft, INPUT);
  pinMode(touchRight, INPUT);

  WiFi.mode(WIFI_STA);
  Serial.print("WiFi Init...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi OK");
}

// Feedback Function - Updated for single motor/buzzer
void triggerFeedback(char type) {

```

```
if (type == 'S') {  
    // Short beep/vib (400ms) - Successful command  
    digitalWrite(vibMotor, HIGH);  
    digitalWrite(buzzer, HIGH);  
    delay(400);  
    digitalWrite(vibMotor, LOW);  
    digitalWrite(buzzer, LOW);  
}  
else if (type == 'E') {  
    // Long beep/vib (800ms) - Error/Wrong command  
    digitalWrite(vibMotor, HIGH);  
    digitalWrite(buzzer, HIGH);  
    delay(800);  
    digitalWrite(vibMotor, LOW);  
    digitalWrite(buzzer, LOW);  
}  
else if (type == 'C') {  
    // 5 short beeps/vibs - Correct Password  
    for (int i = 0; i < 5; i++) {  
        digitalWrite(vibMotor, HIGH);  
        digitalWrite(buzzer, HIGH);  
        delay(400);  
        digitalWrite(vibMotor, LOW);  
        digitalWrite(buzzer, LOW);  
        if (i < 4) delay(200);  
        // Gap between beeps  
    }  
} // <--- THIS CLOSING BRACE WAS MISSING  
else if (type == 'W') {  
    // 5 long beeps/vibs - Wrong Password  
    for (int i = 0; i < 5; i++) {  
        digitalWrite(vibMotor, HIGH);  
        digitalWrite(buzzer, HIGH);  
        delay(800);  
    }  
}
```

```

    digitalWrite(vibMotor, LOW);

    digitalWrite(buzzer, LOW);

    if (i < 4) delay(200);

    // Gap between beeps
}

}

}

void maintainConnection() {
    if (WiFi.status() != WL_CONNECTED) {
        WiFi.disconnect();
        WiFi.begin(ssid, password);
        delay(500);
        return;
    }

    if (!tcpClient.connected()) {
        if (tcpClient.connect(serverIP, 7890)) {
            Serial.println("Reconnected to Server");
        }
    }
}

void loop() {
    maintainConnection();

    // ---- 1. CHECK FOR ALERTS ----

    if (tcpClient.connected() && tcpClient.available()) {
        char c = tcpClient.read();

        if (c == 'F') {
            // Fire alert - start emergency mode
            emergencyMode = true;
            emergencyTimer = millis();

```

```

}
else if (c == 'S' || c == 'E' || c == 'C' || c == 'W') {
    // Feedback from appliance module
    triggerFeedback(c);
}
}

// ---- 2. EMERGENCY MODE ----
if (emergencyMode) {
    // Continuous short beep/vib pattern
    if (millis() - emergencyTimer > 1000) {
        digitalWrite(vibMotor, HIGH);
        digitalWrite(buzzer, HIGH);
        delay(400);
        digitalWrite(vibMotor, LOW);
        digitalWrite(buzzer, LOW);
        delay(600);
        emergencyTimer = millis();
    }

    bool tL = digitalRead(touchLeft);
    bool tR = digitalRead(touchRight);

    // Check for User Input (Any Touch detected)
    if (tL || tR) {
        // Turn off alert during user response
        digitalWrite(vibMotor, LOW);
        digitalWrite(buzzer, LOW);

        // Wait 2 seconds (Blocking) to confirm long press and count fingers
        delay(2000);

        bool finalL = digitalRead(touchLeft);
        bool finalR = digitalRead(touchRight);
    }
}

```

```

    if (finalL && finalR) {
        // BOTH HELD -> Real Emergency -> Send ACK
        if (tcpClient.connected()) tcpClient.print("ACK\n");
        emergencyMode = false;
        return;
    }
    else if (finalL || finalR) {
        // ONLY ONE HELD -> False Alarm -> Send STOP
        if (tcpClient.connected()) tcpClient.print("STOP\n");
        emergencyMode = false;
        return;
    }
    // If neither are held after 2s, continue emergency mode
    emergencyTimer = millis(); // Reset timer
}
return;
}

// ---- 3. INPUT LOGIC ----

bool tL = digitalRead(touchLeft);
bool tR = digitalRead(touchRight);

// A. EXECUTE COMMAND - Check for LONG PRESS on BOTH sensors
if (tL && tR) {
    // Record start time
    unsigned long bothTouchStart = millis();

    // Wait to see if it's a long press (<1 second for command execution)
    delay(500);

    // Check if BOTH still held after 1 second
    bool finalL = digitalRead(touchLeft);
    bool finalR = digitalRead(touchRight);

```

```

if (finalL && finalR) {
  // BOTH held for 1+ seconds -> EXECUTE COMMAND
  if (localBuffer.length() > 0) {
    if (tcpClient.connected()) {
      Serial.print("Sending: ");
      Serial.println(localBuffer);

      tcpClient.print(localBuffer + "\n");
      localBuffer = "";

      // Wait for feedback from server (handled in section 1)
      delay(1000);
    } else {
      triggerFeedback('E'); // Connection error
    }
  } else {
    Serial.println("Buffer Empty");
    triggerFeedback('E'); // Error - no command to send
  }

  // Wait for release
  while(digitalRead(touchLeft) == HIGH || digitalRead(touchRight) == HIGH) {
    delay(10);
  }
  delay(500); // Debounce
}

// If released early, it was accidental - ignore
return;
}

// B. TYPING LOGIC
if ((tL || tR) && (millis() - lastDebounce > 400)) {
  byte pattern = 0;

```

```

for (int i=0; i<6; i++) {
  if (digitalRead(btnPins[i]) == LOW) pattern |= (1 << i);
}

if (pattern > 0) {
  char detectedChar = '?';
  for (int i=0; i<26; i++) {
    if (brailleMap[i] == pattern) {
      if (tR) detectedChar = brailleChars[i];
      else if (tL) detectedChar = (i < 9) ? (i + 1) + '0' : (i == 9 ? '0' : '?');
      break;
    }
  }
}

```

```

if (detectedChar != '?') {
  localBuffer += detectedChar;
  Serial.print("Buf: ");
  Serial.println(localBuffer);
}

```

```

// Short feedback for character entry

```

```

digitalWrite(vibMotor, HIGH);
digitalWrite(buzzer, HIGH);
delay(100);
digitalWrite(vibMotor, LOW);
digitalWrite(buzzer, LOW);

```

```

lastDebounce = millis();

```

```

}

```

```

}

```

```

}

```

```

}

```

```

/*

```

```

* BRAILLE HOME AUTOMATION - APPLIANCE MODULE (UPDATED)

```


- * - Password: 1010 (Touch=Clock, Btn=Data)
- * - Fire Logic:
- * - 'ACK' (Remote Both Sensors) -> Stop Alarm + SEND BLYNK + OPEN DOOR
- * - 'STOP' (Remote One Sensor) -> Stop Alarm + NO BLYNK
- * - Flame Sensor: ACTIVE HIGH
- * - Feedback: 'S'=success, 'E'=error, 'C'=correct password, 'W'=wrong password
- */

```
#define BLYNK_TEMPLATE_ID "TMPL3VTwLUsho"
#define BLYNK_TEMPLATE_NAME "BRAILLE ASSISTIVE HOME"
#define BLYNK_AUTH_TOKEN "xFRDYtkIVA-pcnq9-d_y8FMOUg57N_j2"
```

```
#include <WiFi.h>
#include <ESP32Servo.h>
#include <Wire.h>
#include <Adafruit_SSD1306.h>
#include <BlynkSimpleEsp32.h>
#include <Preferences.h>
```

```
#define RELAY_ON LOW
#define RELAY_OFF HIGH
```

```
// ---- PIN DEFINITIONS ----
#define RELAY_L1 13
#define RELAY_L2 12
#define RELAY_F1_1 14
#define RELAY_F1_2 27
#define RELAY_F1_3 26
#define RELAY_F2_1 2
#define RELAY_F2_2 15
#define RELAY_F2_3 32
#define SERVO_PIN 25
#define FLAME_PIN 4    // ACTIVE HIGH
#define OLED_SDA 21
```

```
#define OLED_SCL 22
#define MAN_SW1 18
#define MAN_SW2 19
#define FAN_BTN1 34
#define FAN_BTN2 35
#define BTN_PASS 5      // Data Bit (Held=1, Released=0)
#define TOUCH_PASS 23   // Clock/Enter (Tap=Bit, Hold=Enter)

const char* router_ssid = "raghav";
const char* router_pass = "12345678";
const char* ap_ssid = "ESP32-Server";
const char* ap_pass = "12345678";

const String CORRECT_PASSWORD = "1010";

WiFiServer tcpServer(7890);
WiFiClient activeClient;
Servo doorServo;
Adafruit_SSD1306 display(128, 64, &Wire, -1);
Preferences preferences;

// ---- GLOBAL VARIABLES ----
bool oledStatus = false;
String inputBuffer = "";
String lastCommand = "None";
bool fireActive = false;
unsigned long lastFireMsg = 0;
unsigned long doorOpenTime = 0;
bool doorAutoClose = false;

// Appliance States
bool l1State = false;
bool l2State = false;
int fan1State = 0;
```

```
int fan2State = 0;

// Button Debouncing/State
int lastSw1State = HIGH;
int lastSw2State = HIGH;
int savedFan1 = 1;
int lastBtn1State = HIGH;
unsigned long btn1PressStart = 0;
bool btn1Active = false;
bool btn1LongPress = false;
unsigned long lastFan1Change = 0;

int savedFan2 = 1;
int lastBtn2State = HIGH;
unsigned long btn2PressStart = 0;
bool btn2Active = false;
bool btn2LongPress = false;
unsigned long lastFan2Change = 0;

// Password Logic Variables
String passInput = "";
unsigned long touchStartTime = 0;
bool touchHandled = false;
unsigned long btnPassStart = 0;
bool btnPassActive = false;

// ---- OLED FUNCTIONS ----
void updateOled(String msg) {
  if (oledStatus) {
    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(WHITE);
    display.setCursor(0, 0);
    display.println(msg);
```

```

    display.display();
}
}

void showStatus() {
    if (oledStatus) {
        display.clearDisplay();
        display.setTextSize(2);
        display.setTextColor(WHITE);

        // Line 1: Last Command
        display.setCursor(0, 0);
        display.print("CMD:");
        if(lastCommand.length() > 5) display.println(lastCommand.substring(0,5));
        else display.println(lastCommand);
        display.drawLine(0, 16, 127, 16, WHITE);

        // Line 2: Lights
        display.setCursor(0, 20);
        display.print("L1:"); display.print(l1State);
        display.print(" L2:"); display.println(l2State);

        // Line 3: Fans
        display.setCursor(0, 40);
        display.print("F1:"); display.print(fan1State);
        display.print(" F2:"); display.println(fan2State);

        display.display();
    }
}

// ---- FAN CONTROL FUNCTIONS ----
void setFan1Speed(int speed) {
    if (speed == 0) {

```

```

digitalWrite(RELAY_F1_1, RELAY_OFF);
digitalWrite(RELAY_F1_2, RELAY_OFF);
digitalWrite(RELAY_F1_3, RELAY_OFF);
} else {
digitalWrite(RELAY_F1_1, (speed==1 || speed==3) ? RELAY_ON : RELAY_OFF);
digitalWrite(RELAY_F1_2, (speed==2 || speed==3) ? RELAY_ON : RELAY_OFF);
digitalWrite(RELAY_F1_3, (speed==4) ? RELAY_ON : RELAY_OFF);
}
if(Blynk.connected()) Blynk.virtualWrite(V3, speed);
}

void setFan2Speed(int speed) {
if (speed == 0) {
digitalWrite(RELAY_F2_1, RELAY_OFF);
digitalWrite(RELAY_F2_2, RELAY_OFF);
digitalWrite(RELAY_F2_3, RELAY_OFF);
} else {
digitalWrite(RELAY_F2_1, (speed==1 || speed==3) ? RELAY_ON : RELAY_OFF);
digitalWrite(RELAY_F2_2, (speed==2 || speed==3) ? RELAY_ON : RELAY_OFF);
digitalWrite(RELAY_F2_3, (speed==4) ? RELAY_ON : RELAY_OFF);
}
if(Blynk.connected()) Blynk.virtualWrite(V4, speed);
}

// ---- SETUP ----
void setup() {
Serial.begin(115200);

// Relays
pinMode(RELAY_L1, OUTPUT); digitalWrite(RELAY_L1, RELAY_OFF);
pinMode(RELAY_L2, OUTPUT); digitalWrite(RELAY_L2, RELAY_OFF);
pinMode(RELAY_F1_1, OUTPUT); digitalWrite(RELAY_F1_1, RELAY_OFF);
pinMode(RELAY_F1_2, OUTPUT); digitalWrite(RELAY_F1_2, RELAY_OFF);
pinMode(RELAY_F1_3, OUTPUT); digitalWrite(RELAY_F1_3, RELAY_OFF);

```

```
pinMode(RELAY_F2_1, OUTPUT); digitalWrite(RELAY_F2_1, RELAY_OFF);  
pinMode(RELAY_F2_2, OUTPUT); digitalWrite(RELAY_F2_2, RELAY_OFF);  
pinMode(RELAY_F2_3, OUTPUT); digitalWrite(RELAY_F2_3, RELAY_OFF);
```

```
// Sensors
```

```
pinMode(FLAME_PIN, INPUT);  
pinMode(MAN_SW1, INPUT_PULLUP);  
pinMode(MAN_SW2, INPUT_PULLUP);  
pinMode(BTN_PASS, INPUT_PULLUP);  
pinMode(TOUCH_PASS, INPUT);  
pinMode(FAN_BTN1, INPUT);  
pinMode(FAN_BTN2, INPUT);
```

```
doorServo.attach(SERVO_PIN);  
doorServo.write(0);
```

```
Wire.begin(OLED_SDA, OLED_SCL);  
if(display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { oledStatus = true; }  
else if(display.begin(SSD1306_SWITCHCAPVCC, 0x3D)) { oledStatus = true; }  
else { oledStatus = false; }
```

```
if (oledStatus) {  
    display.clearDisplay();  
    display.setTextSize(2);  
    display.setTextColor(WHITE);  
    display.setCursor(0, 0);  
    display.println("System\nStarting");  
    display.display();  
}
```

```
// Preferences
```

```
preferences.begin("home_auto", false);  
l1State = preferences.getBool("l1", false);  
l2State = preferences.getBool("l2", false);
```

```

fan1State = preferences.getInt("f1", 0);
fan2State = preferences.getInt("f2", 0);

if (fan1State > 0) savedFan1 = fan1State;
if (fan2State > 0) savedFan2 = fan2State;

digitalWrite(RELAY_L1, l1State ? RELAY_ON : RELAY_OFF);
digitalWrite(RELAY_L2, l2State ? RELAY_ON : RELAY_OFF);
setFan1Speed(fan1State);
setFan2Speed(fan2State);

lastSw1State = digitalRead(MAN_SW1);
lastSw2State = digitalRead(MAN_SW2);

// WiFi
WiFi.mode(WIFI_AP_STA);
WiFi.softAP(ap_ssid, ap_pass);
WiFi.begin(router_ssid, router_pass);

unsigned long s = millis();
while(WiFi.status() != WL_CONNECTED && millis() - s < 5000) { delay(100); }

Blynk.config(BLYNK_AUTH_TOKEN);
if (WiFi.status() == WL_CONNECTED) { Blynk.connect(); }

tcpServer.begin();
showStatus();
}

void loop() {
  Blynk.run();

  // ---- AUTO CLOSE DOOR ----
  if (doorAutoClose && (millis() - doorOpenTime > 5000)) {

```

```

doorServo.write(0);

doorAutoClose = false;

updateOled("Door Closed");

delay(1000);

showStatus();
}

// ---- TCP CLIENT HANDLING ----

if (tcpServer.hasClient()) {
    if (activeClient && activeClient.connected()) {
        WiFiClient newClient = tcpServer.available();
        newClient.stop();
    }
    else {
        activeClient = tcpServer.available();
        Serial.println("Remote Connected");
    }
}

if (activeClient && activeClient.connected()) {
    while (activeClient.available()) {
        char c = activeClient.read();

        if (c == '\n') {
            inputBuffer.trim();
            lastCommand = inputBuffer;
            Serial.println("EXEC: " + inputBuffer);

            char response = 'E';
            bool statusChanged = false;

            // ---- COMMAND PARSING ----

            if (inputBuffer == "L1") {
                l1State = !l1State;

```



```

digitalWrite(RELAY_L1, l1State ? RELAY_ON : RELAY_OFF);
preferences.putBool("l1", l1State);
if(Blynk.connected()) Blynk.virtualWrite(V1, l1State);
response = 'S';
statusChanged = true;
}
else if (inputBuffer == "L2") {
    l2State = !l2State;
    digitalWrite(RELAY_L2, l2State ? RELAY_ON : RELAY_OFF);
    preferences.putBool("l2", l2State);
    if(Blynk.connected()) Blynk.virtualWrite(V2, l2State);
    response = 'S';
    statusChanged = true;
}
else if (inputBuffer.startsWith("F") && inputBuffer.length() >= 3) {
    int r = inputBuffer.charAt(1) - '0';
    int s = inputBuffer.charAt(2) - '0';

    if (r==1) {
        fan1State=s; setFan1Speed(s); preferences.putInt("f1", s);
        if(s>0) savedFan1=s; response='S'; statusChanged = true;
    }
    else if (r==2) {
        fan2State=s; setFan2Speed(s); preferences.putInt("f2", s);
        if(s>0) savedFan2=s; response='S'; statusChanged = true;
    }
}
else if (inputBuffer == "ALL1") {
    l1State = true; l2State = true; fan1State = 4; fan2State = 4;
    digitalWrite(RELAY_L1, RELAY_ON); digitalWrite(RELAY_L2, RELAY_ON);
    setFan1Speed(4); setFan2Speed(4);
    if(Blynk.connected()) {
        Blynk.virtualWrite(V1, 1);
        Blynk.virtualWrite(V2, 1);
    }
}

```

```

    }

    response = 'S';
    statusChanged = true;
}

else if (inputBuffer == "ALL0") {
    l1State = false; l2State = false; fan1State = 0; fan2State = 0;
    digitalWrite(RELAY_L1, RELAY_OFF); digitalWrite(RELAY_L2, RELAY_OFF);
    setFan1Speed(0); setFan2Speed(0);
    if(Blynk.connected()) {
        Blynk.virtualWrite(V1, 0);
        Blynk.virtualWrite(V2, 0);
    }
    response = 'S';
    statusChanged = true;
}

else if (inputBuffer == "OPN") {
    doorServo.write(90);
    doorOpenTime = millis();
    doorAutoClose = true;
    response = 'S';
    updateOled("Door Open");
    delay(2000);
    statusChanged = true;
}

else if (inputBuffer == "CLS") {
    doorServo.write(0);
    doorAutoClose = false;
    response = 'S';
    updateOled("Door Closed");
    delay(2000);
    statusChanged = true;
}

// ---- EMERGENCY: CONFIRMED (ACK) ----

else if (inputBuffer == "ACK") {

```

```
fireActive = false;

response = 'S';

updateOled("Alert\nConfirmed");


// OPEN DOOR AUTOMATICALLY

doorServo.write(90);

doorOpenTime = millis();

doorAutoClose = true;


// SEND BLYNK NOTIFICATION

if(Blynk.connected()) {
  Blynk.logEvent("fire_alert", "Fire Alarm Confirmed by User!");
}

delay(1000);

statusChanged = true;
}

// ---- EMERGENCY: FALSE ALARM (STOP) ----

else if (inputBuffer == "STOP") {
  fireActive = false;

  response = 'S';

  updateOled("False Alarm");

  // NO BLYNK, NO DOOR OPENING

  delay(1000);

  statusChanged = true;
}

activeClient.print(response);

showStatus();

inputBuffer = "";

} else {

  inputBuffer += c;
}
```

```

    }
}

// ---- MANUAL SWITCHES ----

int currentSw1 = digitalRead(MAN_SW1);
if (currentSw1 != lastSw1State) {
    lastSw1State = currentSw1;
    l1State = !l1State;
    digitalWrite(RELAY_L1, l1State ? RELAY_ON : RELAY_OFF);
    preferences.putBool("l1", l1State);
    if(Blynk.connected()) Blynk.virtualWrite(V1, l1State);
    showStatus();
}

int currentSw2 = digitalRead(MAN_SW2);
if (currentSw2 != lastSw2State) {
    lastSw2State = currentSw2;
    l2State = !l2State;
    digitalWrite(RELAY_L2, l2State ? RELAY_ON : RELAY_OFF);
    preferences.putBool("l2", l2State);
    if(Blynk.connected()) Blynk.virtualWrite(V2, l2State);
    showStatus();
}

// ---- PASSWORD LOGIC ----

int touchVal = digitalRead(TOUCH_PASS);
int btnVal = digitalRead(BTN_PASS);

if (btnVal == LOW && !btnPassActive) {
    btnPassActive = true;
    btnPassStart = millis();
}

if (btnVal == LOW && btnPassActive) {

```

```
if (millis() - btnPassStart > 2000) {  
    passInput = "";  
    updateOled("Input\nCLEARED");  
    delay(1000);  
    showStatus();  
    btnPassActive = false;  
}  
}
```

```
if (btnVal == HIGH) btnPassActive = false;
```

```
if (touchVal == HIGH && !touchHandled) {  
    touchStartTime = millis();  
    touchHandled = true;  
}
```

```
if (touchVal == HIGH && touchHandled) {  
    if (millis() - touchStartTime > 2000) {  
        if (fireActive) {  
            fireActive = false;  
            updateOled("Alarm\nSTOPPED");  
            delay(1000);  
            showStatus();  
        }  
        else {  
            if (passInput == CORRECT_PASSWORD) {  
                updateOled("Access\nGRANTED");  
                if (activeClient && activeClient.connected()) activeClient.print('C');  
                passInput = "";  
            } else {  
                updateOled("Access\nDENIED");  
                if (activeClient && activeClient.connected()) activeClient.print('W');  
                passInput = "";  
                delay(2000);  
            }  
        }  
    }  
}
```

```

    }
    showStatus();
}
while(digitalRead(TOUCH_PASS) == HIGH) { delay(10); }
}
}

if (touchVal == LOW && touchHandled) {
    if (millis() - touchStartTime < 1000) {
        if (btnVal == LOW) passInput += "1";
        else passInput += "0";
        updateOled("PW: " + passInput);
    }
    touchHandled = false;
}

// ---- FIRE SENSOR LOGIC (ACTIVE HIGH) ----
if (digitalRead(FLAME_PIN) == HIGH) {
    if (!fireActive) {
        fireActive = true;
        updateOled("!! FIRE !!");
    }

    if (millis() - lastFireMsg > 1000) {
        lastFireMsg = millis();
        if (activeClient && activeClient.connected()) activeClient.print('F');
    }
}

// ---- FAN BUTTONS ----
int f1Btn = digitalRead(FAN_BTN1);
if (f1Btn == LOW && lastBtn1State == HIGH) {
    btn1PressStart = millis();
    btn1Active = true;

```

```

    btn1LongPress = false;
}

if (f1Btn == LOW && btn1Active) {
    if (millis() - btn1PressStart > 500) {
        btn1LongPress = true;
        if (millis() - lastFan1Change > 500) {
            lastFan1Change = millis();
            if (fan1State == 0) fan1State = 1;
            else {
                fan1State++;
                if (fan1State > 4) fan1State = 1;
            }
            setFan1Speed(fan1State);
            preferences.putInt("f1", fan1State);
            if(fan1State>0) savedFan1=fan1State;
            showStatus();
        }
    }
}

if (f1Btn == HIGH && lastBtn1State == LOW) {
    if (!btn1LongPress) {
        if (fan1State > 0) {
            savedFan1 = fan1State;
            fan1State = 0;
        } else {
            fan1State = (savedFan1 > 0) ? savedFan1 : 1;
        }
        setFan1Speed(fan1State);
        preferences.putInt("f1", fan1State);
        showStatus();
    }
    btn1Active = false;
}

```

```
}  
lastBtn1State = f1Btn;  
  
int f2Btn = digitalRead(FAN_BTN2);  
if (f2Btn == LOW && lastBtn2State == HIGH) {  
    btn2PressStart = millis();  
    btn2Active = true;  
    btn2LongPress = false;  
}
```

```
if (f2Btn == LOW && btn2Active) {  
    if (millis() - btn2PressStart > 500) {  
        btn2LongPress = true;  
        if (millis() - lastFan2Change > 500) {  
            lastFan2Change = millis();  
            if (fan2State == 0) fan2State = 1;  
            else {  
                fan2State++;  
                if (fan2State > 4) fan2State = 1;  
            }  
            setFan2Speed(fan2State);  
            preferences.putInt("f2", fan2State);  
            if(fan2State>0) savedFan2=fan2State;  
            showStatus();  
        }  
    }  
}
```

```
if (f2Btn == HIGH && lastBtn2State == LOW) {  
    if (!btn2LongPress) {  
        if (fan2State > 0) {  
            savedFan2 = fan2State;  
            fan2State = 0;  
        } else {
```



```
        fan2State = (savedFan2 > 0) ? savedFan2 : 1;
    }
    setFan2Speed(fan2State);
    preferences.putInt("f2", fan2State);
    showStatus();
}
btn2Active = false;
}
lastBtn2State = f2Btn;
}
```